T3000 API & Protocol

Message 12: GET_SELECTED_DEVICE_INFO · built Nov 13, 2026

Message 12: GET_SELECTED_DEVICE_INFO

The GET_SELECTED_DEVICE_INFO message retrieves information about the currently selected device in T3000.

When to Use:

- Getting current device context
- Determining active panel details
- Synchronizing frontend with backend device selection

Overview

Action: `GET_SELECTED_DEVICE_INFO` (12) **Direction:** Frontend → Backend **Location:** BacnetWebView.cpp line 3028 **Purpose:** Get currently selected device information

Request Format

JSON Structure:

```JSON
{
  "action": "GET_SELECTED_DEVICE_INFO"
}
```

No parameters required

Response Format

JSON Structure:

JSON

```
{
  "action": "GET_SELECTED_DEVICE_INFO_RES",
  "data": {
    "product_id": 19,
    "panel_id": 0,
    "serial_number": 237219
  }
}
```

Response Fields

Field	Type	Description
`product_id`	number	Device product type ID
`panel_id`	number	Panel ID (0-254)
`serial_number`	number	Device serial number

Product IDs

Common product IDs:

- **19**: T3-BB (BACnet Building Controller)
- **PM_T322AI**: T3-22AI (22 Universal Inputs)
- **PM_T38AI8AO6DO**: T3-8O (8 Outputs)
- See `defines.h` for complete list

Implementation

Location: BacnetWebView.cpp line 3028

C++

```
case GET_SELECTED_DEVICE_INFO:
{
    Json::Value tempjson;
    tempjson["action"] = "GET_SELECTED_DEVICE_INFO_RES";
    tempjson["data"]["product_id"] = g_selected_product_id;
    tempjson["data"]["panel_id"] = bac_gloab_panel;
    tempjson["data"]["serial_number"] = g_selected_serialnumber;

    const std::string output = Json::writeString(builder, tempjson);
    CString temp_cs(output.c_str());
    outmsg = temp_cs;
}
```

Frontend Usage

Get Current Device

TYPESCRIPT

```typescript
const getCurrentDevice = (): Promise<DeviceInfo> => {
  return new Promise((resolve, reject) => {
    const message = {
      action: 'GET_SELECTED_DEVICE_INFO'
    };

    sendMessage(JSON.stringify(message));

    const handler = (data: string) => {
      const response = JSON.parse(data);

      if (response.action === 'GET_SELECTED_DEVICE_INFO_RES') {
        resolve({
          productId: response.data.product_id,
          panelId: response.data.panel_id,
          serialNumber: response.data.serial_number
        });

        webSocket.off('message', handler);
      }
    };

    webSocket.on('message', handler);

    // Timeout after 5 seconds
    setTimeout(() => {
      webSocket.off('message', handler);
      reject(new Error('Timeout waiting for device info'));
    }, 5000);
  });
};

// Usage
const device = await getCurrentDevice();
console.log('Current device:', device);
```

Device Context Provider

TYPESCRIPT

```typescript
const DeviceContext = React.createContext<DeviceInfo | null>(null);

export const DeviceProvider: React.FC = ({ children }) => {
  const [device, setDevice] = useState<DeviceInfo | null>(null);

  useEffect(() => {
    // Get device info on mount
    getCurrentDevice().then(setDevice);
  }, []);

  return (
    <DeviceContext.Provider value={device}>
      {children}
    </DeviceContext.Provider>
  );
};

// Usage in components
const MyComponent = () => {
  const device = useContext(DeviceContext);

  if (!device) return <div>Loading device info...</div>;

  return (
    <div>
      <h3>Device: {device.serialNumber}</h3>
      <p>Panel: {device.panelId}</p>
    </div>
  );
};
```

Sync on Device Change

TYPESCRIPT

```typescript
const syncDeviceInfo = async () => {
  try {
    const device = await getCurrentDevice();

    // Update application state
    store.dispatch(setCurrentDevice(device));

    // Refresh data for new device
    await loadPanelData(device.panelId);
  } catch (error) {
    console.error('Failed to sync device:', error);
  }
};

// Call when user changes device in T3000
window.addEventListener('deviceChanged', syncDeviceInfo);
```

Use Cases

Device-Specific UI

TYPESCRIPT

```typescript
const DevicePanel: React.FC = () => {
  const [device, setDevice] = useState<DeviceInfo | null>(null);

  useEffect(() => {
    getCurrentDevice().then(setDevice);
  }, []);

  if (!device) return <Spinner />;

  return (
    <div>
      <h2>Panel {device.panelId}</h2>
      <p>Serial: {device.serialNumber}</p>
      <p>Type: {getProductName(device.productId)}</p>
    </div>
  );
};
```

Request Validation

TYPESCRIPT

```typescript
const loadData = async (requestedPanelId: number) => {
  const device = await getCurrentDevice();

  // Verify requested panel matches current device
  if (requestedPanelId !== device.panelId) {
    throw new Error(`Panel mismatch: ${requestedPanelId} vs ${device.panelId}`);
  }

  // Proceed with data load
  await fetchPanelData(requestedPanelId);
};
```

Global Variables

The response uses these C++ global variables:

- `g_selected_product_id` - Current product type
- `bac_gloab_panel` - Active panel ID
- `g_selected_serialnumber` - Device serial number

These are set when user selects a device in T3000 tree view.

See Also

- GET_PANELS_LIST - List all available panels
- Platform Overview - Architecture